TILED MAT MUL

08 June 2025 13:42

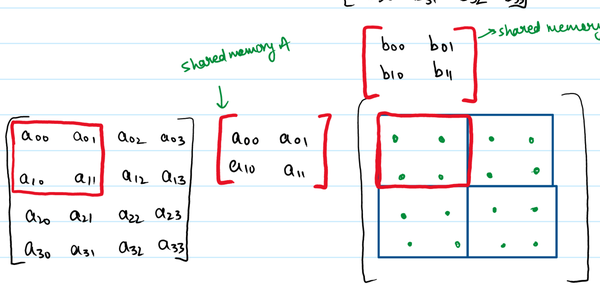
→ A unique shared memory for each block

→ Shared memory of block can be only accessed by the threads in the block

Here N=4, so A & B both are NxN or 4x4 matrices

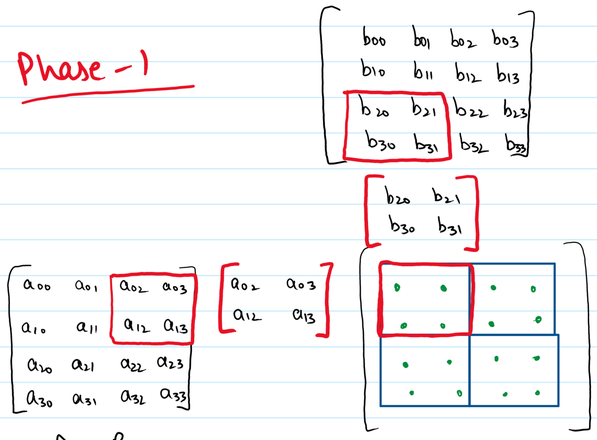
$$\begin{bmatrix} b_{00} & b_{01} & b_{02} & b_{03} \\ b_{10} & b_{11} & b_{12} & b_{13} \\ b_{20} & b_{21} & b_{22} & b_{23} \\ b_{30} & b_{31} & b_{32} & b_{33} \end{bmatrix}$$

Phase 0



→ C is also NxN or 4x4, it has a 2x2 grid, where each thread block is 2x2

Phase - 1

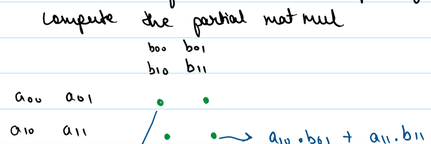


Dry Run

For block '0' we see that there are 2 phases ⇒

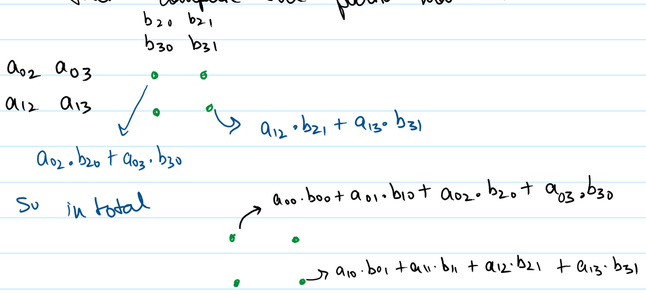
In phase 0

the threads of block 0 load a part of A & B, then compute the partial mat mul



In phase 1

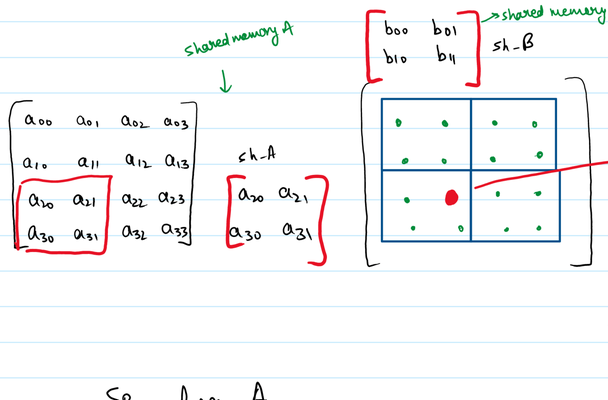
the threads of block 1 load next part of A & B, then compute the partial mat mul



So in this way for 4 elements of 'C' we are done & it took only 4 memory calls whereas in Naive mat mul it would take 8 calls

↳ 4x2 (2 calls → 1 Row, 1 Col for each element)

$$\begin{bmatrix} b_{00} & b_{01} & b_{02} & b_{03} \\ b_{10} & b_{11} & b_{12} & b_{13} \\ b_{20} & b_{21} & b_{22} & b_{23} \\ b_{30} & b_{31} & b_{32} & b_{33} \end{bmatrix}$$



So for A

i = Row index

Tile width * phase + tx = Column Index

$$sh\text{-}A[ty][tx] = A[\, i \times N + \text{Tile width} \times \text{phase} + tx \,]$$

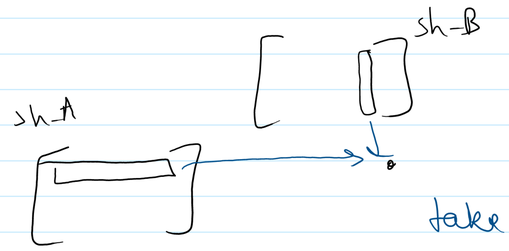
Similarly for B

Tile width x phase + ty = Row index

j = Col index

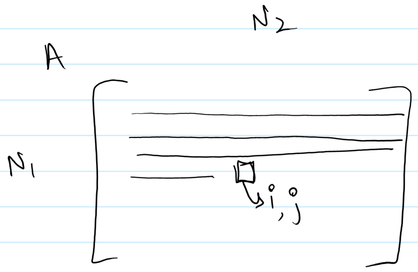
$$sh\text{-}B[ty][tx] = B[\,(\text{Tile width} \times \text{phase} + ty)N + j\,]$$

Now we have sh-A & sh-B for phase 0



take partial dot product & then move to next phase

for Rectangular matrix



$N_1 \times N_2$

$$A[i][j] = A[\, i \times N2 + j \,]$$